Área de Arquitectura y Tecnología de Computadores (ARCOS)

Universidad Carlos III de Madrid

SISTEMAS OPERATIVOS

Práctica 1. Llamadas al sistema operativo

Grado en Ingeniería Informática
Grado en Matemática Aplicada y Computación
Doble Grado en Ingeniería Informática y Administración de Empresas

Curso 2025/2026

Tabla de contenido

1 Enunciado de la Práctica..... 3
1.1 Descripción de la Práctica..... 3
1.1.1 mycalc..... 3
1.1.2 mydu..... 5
1.1.3 Código Fuente de apoyo..... 8
1.1.4 Comprobador de entrega proporcionado..... 9
1.2 Consideraciones sobre el uso de IA..... 10
2 Entrega..... 10
2.1 Plazo de Entrega..... 10
2.2 Compilación y pruebas..... 10
2.3 Procedimiento de entrega de las prácticas..... 10
2.4 Documentación a Entregar..... 11
3 Normas..... 13
4 Anexo (Llamadas al Sistema)..... 14
4.1 Llamadas al sistema relacionadas con archivos..... 14
4.2 Manual (man function)..... 15
5 Bibliografía..... 17

1 Enunciado de la Práctica

Esta práctica permite al alumno familiarizarse con las llamadas al sistema operativo siguiendo el estándar POSIX. Unix permite efectuar llamadas al sistema directamente desde un programa realizado en un lenguaje de alto nivel, en particular en lenguaje C. La mayor parte de las entradas/salidas (E/S) sobre ficheros en Unix pueden realizarse utilizando solamente pocas llamadas: *open, creat, read, write, lseek* y *close*; y lo mismo se puede hacer sobre directorios: *opendir, readdir*, etc.

Para el kernel del sistema operativo, todos los archivos abiertos son identificados por medio de descriptores de fichero. Un descriptor de fichero es un entero no negativo. Cuando abrimos ("*open*") un archivo que ya existe, el kernel devuelve un descriptor de fichero al proceso. Cuando queremos leer o escribir desde/en un archivo, identificamos el archivo con el descriptor de fichero que fue devuelto por la llamada anteriormente descrita.

Cada archivo abierto tiene una posición de lectura/escritura actual ("**current file offset**"). Está representado por un entero no negativo que mide el número de bytes desde el comienzo del archivo. Las operaciones de lectura y escritura comienzan normalmente en la posición actual y provocan un incremento en dicha posición, igual al número de bytes leídos o escritos. Por defecto, esta posición es inicializada a 0 cuando se abre un archivo, a menos que se especifique la opción O_APPEND. La posición actual (current offset) de un archivo abierto puede cambiarse explícitamente utilizando la llamada al sistema *lseek*.

Por otro lado, para poder trabajar con la información en forma de directorios (no de ficheros), también existen llamadas al sistema destinadas a tal propósito. Al igual que sucede con los ficheros, se tiene un puntero de desplazamiento para las entradas de los directorios, y se pueden recorrer obteniendo distintos datos. De esta forma, se puede recorrer el árbol del sistema de ficheros sin necesidad de bibliotecas o componentes externos.

En el apartado de Anexos podrás encontrar más información sobre estas funciones para realizar llamadas al sistema.

1.1 Descripción de la Práctica

Se deben implementar los programas **mycalc** y **mydu** en C, utilizando las llamadas al sistema descritas anteriormente y los ficheros de código fuente de apoyo mycalc.c y mydu.c

1.1.1 mycalc

El primer programa, **mycalc**, realiza las funciones de una calculadora simple. Tiene dos modos de funcionamiento: **Calculadora interactiva** e **Historial**.

El programa en modo "**calculadora interactiva**" recibirá 3 argumentos:

1) un número entero,
2) un carácter que defina una operación y
3) un segundo número entero.

Como idea general, el programa debe calcular la operación insertada con los valores especificados y mostrar el resultado por pantalla. Las operaciones permitidas son: suma (+), resta (-), multiplicación (x) y división (/).

Además, cada operación realizada con éxito deberá ser almacenada en un fichero de log llamado "**mycalc.log**", que contendrá una operación con su resultado en cada línea.

El programa en modo "**historial**" recibirá dos argumentos:

1. "-b" para indicar el modo histórico y
2. un número entero que identificará la línea de la operación que se quiere consultar en el fichero de log.

Esta funcionalidad debe leer la línea completa de la operación desde el fichero de log y mostrarla por pantalla. No es necesario recalcular, únicamente mostrar.

Uso 1 (calculadora interactiva): `./mycalc <num1> <operación> <num2>`
Uso 2 (historial): `./mycalc -b <num operación>`

- **Requisitos:**
  - El programa **no debe** hacer uso de la biblioteca "**stdio.h"** y debe realizar únicamente uso de llamadas al sistema.
  - Los errores detectados deben mostrar mensajes significativos por la salida de error.
  - Los errores detectados no se guardan en el log de salida (únicamente operaciones correctas).
  - La información acerca de las operaciones y su resultado debe mostrarse por la salida estándar.
  - El programa debe devolver -1 ante cualquier error detectado.
  - El programa debe devolver -1 si la operación no está definida en los símbolos indicados.
  - El programa debe devolver -1 si el flag de "modo histórico" es incorrecto.
  - El programa debe devolver 0 si todo funcionó correctamente.

- **Sugerencia de prueba:** Contrastar que los mensajes se muestran por la salida correcta. Contrastar que la línea indicada en el modo histórico obtiene la línea deseada.

Ejemplo de salida del programa en modo calculadora:

```shell
./mycalc 5242 x 3442
Operación: 5242 x 3442 = 18042964
./mycalc 5242 x
Uso (1): ./mycalc <num1> <operacion (+|-|x|/)> <num2>
Uso (2): ./mycalc -b <num operacion>
./mycalc 8374 + 1121
Operación: 8374 + 1121 = 9495
./mycalc 8374 / 0
Error: División por cero
```

Considerando el siguiente **ejemplo de fichero de log:**

```shell
Operación: 5242 x 3442 = 18042964
Operación: 8374 + 1121 = 9495
Operación: 3 x 21 = 63
Operación: 3 / 21 = 0
Operación: 21 / 3 = 7
Operación: 5 x 4 = 20
Operación: 5 x 34 = 170
```

Se obtiene el **ejemplo de salida del programa en modo histórico:**

```shell
./mycalc -b 1
Línea 1: Operación: 5242 x 3442 = 18042964

./mycalc -b 4
Línea 4: Operación 3 / 21 = 0

./mycalc -b -1
Error: El número de línea no es válido

./mycalc -b 10
Error: El número de línea no es válido
```

1.1.2 mydu

El segundo programa, **mydu**, realizará una operación de cálculo del tamaño que ocupa un directorio y todo su contenido (de forma recursiva). Esto quiere decir que, si el directorio indicado tiene subdirectorios (y estos, a su vez, etc.), se debe acumular el tamaño de todos los niveles hasta obtener el tamaño total (en **KB**).

Este programa tendrá tres modos de funcionamiento:

1. Cálculo del tamaño de un directorio pasado como argumento.
2. Cálculo del tamaño del directorio actual (desde donde se ejecuta el programa), en caso de que no se pase el argumento.
3. Lectura de un fichero binario con el historial de los directorios analizados. Cada vez que se ejecuta "./mydu [directorio]", los resultados que se escriben por pantalla, se escriben también en un fichero **binario (no texto)**, llamado "**mydu.bin**". Para ello, se recomienda crear una estructura con dos campos (tamaño y path) y que sea dicha estructura la que se escriba en el fichero. Al ejecutar el programa, se puede indicar el flag "-b" que indica que se quiere mostrar por pantalla el contenido del fichero binario en modo legible.

NOTA

Para este ejercicio se permite el uso de funciones de biblioteca como "printf" y "perror" para mostrar mensajes por pantalla, tanto por la salida estándar como por la salida de error, pero la escritura/lectura de ficheros debe realizarse con llamadas al sistema (open, read, write, lseek, close).

Uso 1: `./mydu [<directorio>]`
Uso 2: `./mydu [-b]`

- **Requisitos:**
  - El programa debe devolver -1 si se detecta cualquier condición de error.
  - Los mensajes de error deben ser mostrados por la salida de error, mientras que la información correcta debe mostrarse por la salida estándar.
  - Si es correcto, debe devolver 0.
  - Los directorios listados con tamaño deben encadenar la ruta completa desde el directorio indicado (no listar únicamente el nombre del fichero).
  - Los ficheros regulares NO son argumentos válidos. Si se detecta un fichero, se mostrará por pantalla su nombre y un mensaje de error.
  - El fichero binario debe llamarse "mydu.bin"
- **Sugerencia de prueba:** Contrastar que el cálculo realizado es igual al que proporciona el comando "du" en el terminal sobre el mismo directorio. Realiza tus pruebas finales en el servidor Guernika del Laboratorio de Informática de la UC3M.

Estructura de directorio de ejemplo:

```
folder1
     |_ file(s)

     |_ folder2
```

```
        |_ file(s)
    |_ folder3
        |_ file(s)
```

Ejemplo de salida del programa en Guernika:

```shell
./mydu
3744   folder1/folder2
3168   folder1/folder3
8352   folder1
8276   .

./mydu .
3744   folder1/folder2
3168   folder1/folder3
8352   folder1
8412    .
# Observe que el tamaño de "." ha incrementado porque el fichero binario "mydu.bin"
se ha creado en la primera ejecución.

./mydu folder1/folder2
3744   folder1/folder2

./mydu mydu.bin
mydu.bin: No es un directorio
```

Ejemplo de salida del programa en modo lectura de binario (tras el proceso anterior):

```shell
./mydu -b

--- Contenido del archivo binario ---
3744    ./folder1/folder2
3168    ./folder1/folder3
8352    ./folder1
8276    .
3744    ./folder1/folder2
3168    ./folder1/folder3
8352    ./folder1
8412    .
3744    folder1/folder2
```

1.1.3 Código Fuente de apoyo

Para facilitar la realización de esta práctica se dispone del fichero **ssoo_p1_llamadas_2026.zip** que contiene código fuente de apoyo. Para extraer su contenido, ejecutar el siguiente comando:

```
unzip ssoo_p1_llamadas_2026.zip
```

Al extraer su contenido, se crea el directorio p1_llamadas/, donde se debe desarrollar la práctica. Dentro de este directorio se habrán incluido los siguientes ficheros:

- Makefile
  NO debe ser modificado. Fichero fuente para la herramienta make. Con él se consigue la recompilación automática solo de los ficheros fuente que se modifiquen. Utilice "make" para compilar los programas, y "make clean" para eliminar los archivos compilados.

- mycalc.c
  Debe modificarse. Fichero fuente de C donde los alumnos deberán codificar el programa **mycalc**.

- mydu.c
  Debe modificarse. Fichero fuente de C donde los alumnos deberán codificar el programa **mydu**.

- autores.txt
  Debe modificarse. Fichero en formato de texto (.txt) donde se deben incluir los autores de la práctica (uno por línea). Formato (sin tildes):
  NIA1,Apellido(s),Nombre(s)
  NIA2,Apellido(s),Nombre(s)
  NIA3,Apellido(s),Nombre(s)

```
Textproto
Ejemplo válido:
100499455,Apellido1 Apellido2,Nombre
100499455,Apellido1 Apellido2,Nombre Compuesto
```

```
Textproto
Ejemplo NO válido:
,Apellido1 Apellido2,Nombre1 (no incluye el NIA)
(línea en blanco)
Apellido1 Apellido2,Nombre Compuesto,100499455
499455,Apellido1 Apellido2,Nombre (NIA incompleto)
```

1.1.4 Comprobador de entrega proporcionado

Se proporciona a los alumnos un script en Python (versión 3) "**comprobador_ssoo_p1.py**" que verifica que el formato del entregable de la práctica es el correcto. El script se encarga de comprobar que la entrega sigue las especificaciones de nombrado, está bien comprimida e incluye lo especificado, **y compila correctamente**.

Requisitos

El siguiente software deberá estar instalado en el entorno en donde se vaya a ejecutar el comprobador:

- Python 3
- Make
- Unzip

En el servidor Guernika del Laboratorio de Informática de la UC3M (https://www.lab.inf.uc3m.es/servicios/aulas-virtuales-del-laboratorio/) ya se encuentran instaladas estas dependencias. En otro caso, deberás ejecutar el siguiente comando en tu ordenador si trabajas con una máquina virtual, WSL o Docker:

```shell
sudo apt-get update && apt-get install -y make unzip
```

Uso

Se asume que el comando "python3" existe. En caso de que sea utilice un alias, conviene sustituir "python3" por "python" en el siguiente comando para ejecutar el comprobador:

```shell
python3 comprobador_ssoo_p1.py <entregable.zip>
```

Siendo **entregable.zip** el fichero que se va a entregar por Aula Global (ver el apartado de Entrega). Por ejemplo:

```shell
python3 comprobador_ssoo_p1.py ssoo_p1_100254896_100047014.zip
```

El comprobador imprimirá mensajes por pantalla indicando si el formato es o no correcto.

NOTA

El uso de este comprobador es obligatorio antes de realizar la entrega, puesto que verifica la correcta compilación de los programas. Durante la corrección, toda entrega que no compile obtendrá una calificación directa de 0 puntos.

1.2 Consideraciones sobre el uso de IA

Si bien la utilización de herramientas de IA está permitido, se recomienda encarecidamente evitar su uso por un motivo puramente pedagógico. Cabe recordar que, aunque no se va a comprobar el empleo de este tipo de herramientas, hay dos motivos por los que es mejor no basarse en ellas en esta asignatura:

1. Aprender a programar en C/C++ y otros lenguajes, se aprende programando. Si se delega esa tarea, difícilmente se va a poder realizar después un programa sin ayuda.
2. En esta asignatura se realiza un examen parcial y uno de evaluación final en el que, históricamente, se pide realizar programas en C sobre el papel. Si no se tiene suficiente dominio, no se podrán realizar correctamente estas pruebas.

2 Entrega

Las prácticas se realizarán en equipos de **máximo 3 integrantes** pertenecientes al **mismo grupo reducido**.

2.1 Plazo de Entrega

La fecha límite de entrega de la práctica en AULA GLOBAL será el **6 de marzo de 2026** hasta las 23:55 h.

2.2 Compilación y pruebas

El desarrollo de esta práctica puede ser realizado en cualquiera de las alternativas que se han identificado en la asignatura: Linux nativo, máquina virtual, WSL, o Docker containers. Sin embargo, **para su entrega final, la compilación y pruebas deben ser ejecutadas en el servidor Guernika del Laboratorio de Informática de la UC3M** (https://www.lab.inf.uc3m.es/servicios/aulas-virtuales-del-laboratorio/). En su página web se incluye un apartado de apertura de cuenta para el acceso al servidor.

Una vez que se dispone de cuenta de acceso, las pruebas y compilación final deben realizarse en dicho entorno, puesto que es donde se realizarán las correcciones. Cualquier práctica que no haya realizado la compilación y pruebas en Guernika corre el riesgo de tener algún error que impida su correcta valoración por parte de los profesores.

2.3 Procedimiento de entrega de las prácticas

La entrega de las prácticas ha de realizarse de forma electrónica y por **un único integrante del grupo**. En AULA GLOBAL se habilitarán unos enlaces a través de los cuales se podrá realizar la

entrega de las prácticas. En concreto, **se habilitará un entregador para el código de la práctica, y otro de tipo TURNITIN** para la memoria de la práctica.

2.4 Documentación a Entregar

Se debe entregar un archivo comprimido en formato zip con el nombre:

ssoo_p1_NIA1_NIA2_NIA3.zip

Con los NIAs de los integrantes del grupo. En caso de realizar la práctica en solitario, el formato será **ssoo_p1_NIA.zip**. **El archivo zip se entregará en el entregador correspondiente al código de la práctica**.
El archivo debe contener:

- **Makefile**
- **mydu.c**
- **mycalc.c**
- **autores.txt: Fichero en formato de texto (.txt) donde se deben incluir los autores de la práctica (uno por línea). Formato (sin tildes) y como se ha indicado anteriormente.**

NOTA

Para comprimir dichos ficheros y ser procesados de forma correcta por el probador proporcionado, se recomienda utilizar el siguiente comando:

```
zip ssoo_p1_NIA1_NIA2_NIA3.zip Makefile mycalc.c mydu.c autores.txt
```

La memoria se entregará en formato PDF en un fichero llamado:

ssoo_p1_NIA1_NIA2_NIA3.pdf

Solo se corregirán y calificarán memorias en formato PDF. Tendrá que contener al menos los siguientes apartados:

- **Descripción del código** detallando las principales funciones implementadas. NO incluir código fuente de la práctica en este apartado. Cualquier código será automáticamente ignorado.
- **Batería de pruebas** utilizadas y resultados obtenidos. Se dará mayor puntuación a pruebas avanzadas, casos extremos y, en general, a aquellas pruebas que garanticen el correcto funcionamiento de la práctica en todos los casos. Hay que tener en cuenta:
  1. Que un programa compile correctamente y sin advertencias (warnings) no es garantía de que funcione correctamente.
  2. Evite pruebas duplicadas que evalúan los mismos flujos del programa. La puntuación de este apartado no se mide en función del número de pruebas, sino del grado de cobertura

de las mismas. Es mejor pocas pruebas que evalúan diferentes casos que muchas pruebas que evalúan siempre el mismo caso.**Conclusiones** y problemas encontrados, cómo se han solucionado, y opiniones personales.

Se puntuarán también los siguientes aspectos relativos a la presentación de la práctica:

- Debe contener portada, con los autores de la práctica y sus NIAs.
- Debe contener índice de contenidos.
- La memoria debe tener números de página en todas las páginas (menos la portada).

El archivo PDF se entregará en el entregador correspondiente a la memoria de la práctica (entregador TURNITIN).

NOTA: Es posible entregar el código de la práctica tantas veces como se quiera dentro del plazo de entrega, siendo la última entrega realizada la versión definitiva.

Importante: Si se usan herramientas de IA y las soluciones de varios grupos coinciden, se considerará copia.

LA MEMORIA DE LA PRÁCTICA ÚNICAMENTE SE PODRÁ ENTREGAR UNA ÚNICA VEZ A TRAVÉS DE TURNITIN

3 Normas

1. Las prácticas que **no compilen** o que no se ajusten a la funcionalidad y requisitos planteados, obtendrán una calificación de 0.

2. **Los programas que utilicen funciones de biblioteca (fopen, fread, fwrite, etc.) o similares, en vez de llamadas al sistema, obtendrán una calificación de 0. Tampoco se permite utilizar sentencias o funciones como goto**.

3. Se prestará especial atención a detectar funcionalidades copiadas entre dos prácticas. En caso de encontrar implementaciones comunes en dos prácticas, los alumnos involucrados (copiados y copiadores) perderán las calificaciones obtenidas por evaluación continua.

4. Los programas deben compilar sin ***warnings***.

5. Los programas deberán funcionar bajo un sistema Linux, no se permite la realización de la práctica para sistemas Windows. Además, para asegurarse del correcto funcionamiento de la práctica, deberá chequearse su compilación y ejecución en el servidor Guernika (https://www.lab.inf.uc3m.es/servicios/aulas-virtuales-del-laboratorio/). Si el código presentado no compila o no funciona sobre esta plataforma, la implementación no se considerará correcta.

6. Un programa no comentado obtendrá una **calificación muy baja**.

7. La entrega de la práctica se realizará a través de Aula Global, tal y como se detalla en el apartado Entrega de este documento. No se permite la entrega a través de correo electrónico sin autorización previa.

8. Se debe respetar en todo momento el formato de la entrada y salida que se indica en cada programa a implementar.

9. Se debe realizar un control de errores en cada uno de los programas, **más exhaustivo de lo solicitado explícitamente en cada apartado**.

Los programas entregados que no sigan estas normas no se considerarán aprobados.

4 Anexo (Llamadas al Sistema)

Las llamadas al sistema proporcionan la interfaz entre el sistema operativo y un programa en ejecución. UNIX permite efectuar llamadas al sistema directamente desde un programa realizado en un lenguaje de alto nivel, en particular en lenguaje C, en cuyo caso las llamadas se asemejan a llamadas a funciones, tal y como si estuvieran definidas en una biblioteca estándar. El formato general de una llamada al sistema es:

status = **funcion_estandar** (arg1, arg2,.....)

En caso de realizar una llamada sin éxito, devolvería en la variable status el valor -1. En la variable global *errno* se coloca el número de error, con el cual podemos obtener la asociación del error con lo que realmente ha ocurrido en el fichero errno.h, (contenido en la ruta: /usr/src. En Linux: /usr/src/linux/include/asm/errno.h).

4.1 Llamadas al sistema relacionadas con archivos

*int **open**(const char *path, int flag, ...)*

Abre o crea un fichero especificado por *path*. El fichero abierto puede utilizarse para lectura, escritura, o ambas, en función de lo especificado por *flag*. Devuelve un descriptor de fichero que se puede utilizar para la lectura o escritura en el archivo.
Más ayuda en: man 2 open

*int **close**(int fildes)*

Cierra un archivo abierto anteriormente asociado al descriptor *fildes*.
Si n = -1 →Error al cerrar el fichero.
Más ayuda en: man 2 close

*ssize_t **read**(int fildes, void *buf, size_t nbyte)*

Intenta leer de un archivo (cuyo descriptor de fichero *fildes* se obtuvo de abrirlo) tantos bytes como indica *nbyte*, colocando la información leída a partir de la dirección de memoria *buf*. Devuelve el número de bytes leídos (que puede ser menor o igual a *nbyte*).
Si retorno = 0 → Fin de fichero (EOF).
Si retorno = -1 → Error de lectura.
Más ayuda en: man 2 read

*ssize_t **write**(int fildes, const void * buf, size t nbyte)*

Intenta escribir en un archivo (cuyo descriptor de fichero *fildes* se obtuvo de abrirlo) tantos bytes como indica *nbyte*, tomándolos de la dirección de memoria indicada en *buf*. Devuelve el número de bytes que realmente se han escrito (que puede ser menor o igual a *nbyte*).
Si retorno = -1 → Error de escritura.
Cada *write* o *read* actualiza automáticamente la posición actual del fichero que se usa para determinar la posición en el archivo del siguiente *write* o *read*.
Más ayuda en: man 2 write.

*off_t **lseek**(int fildes, off t offset, int whence)*

Modifica el valor del apuntador del descriptor *fildes* en el archivo, a la posición explícita en *offset* a partir de la referencia impuesta en *whence*, de forma que las llamadas *write* o *read* pueden iniciarse en cualquier parte del archivo.

- Si retorno = -1 → Error de posicionamiento.

El parámetro *whence* puede tomar los siguientes valores:

- SEEK_SET → desde el principio del fichero.
- SEEK_CUR → desde la posición actual.
- SEEK_END → desde el final del fichero.

El parámetro *offset* se expresa en bytes, y toma un valor positivo o negativo. Por ejemplo:

```
Shell
abcdefghi
```

```
C/C++
// El descriptor del fichero abierto "fd" es 5.
lseek(5,4,SEEK_SET) // Al avanzar 4 bytes, la siguiente lectura sería la "e".
```

Más ayuda en: man 2 lseek

4.2 Manual (man function)

man es el paginador del manual del sistema, es decir, permite buscar información sobre un programa, una utilidad o una función. Véase el siguiente ejemplo:

man [seccion] open

Una página de manual tiene varias partes. Estas están etiquetadas como NOMBRE, SINOPSIS, DESCRIPCION, OPCIONES, FICHEROS, …. En la etiqueta de SINOPSIS se recogen las librerías (identificadas por la directiva #include) que se deben incluir en el programa en C del usuario para poder hacer uso de las funciones correspondientes. **Para salir de la página mostrada, basta con pulsar la tecla 'q'.** Las formas más comunes de usar man son las siguientes:

- **man sección elemento**: Presenta la página de elemento disponible en la sección del manual.
- **man -a elemento**: Presenta, secuencialmente, todas las páginas de elemento disponibles en el manual. Entre página y página se puede decidir saltar a la siguiente o salir del paginador completamente.
- **man -k palabra-clave**: Busca la palabra-clave entre las descripciones breves y las páginas de manual y presenta todas las que casen.

5 Bibliografía

- El lenguaje de programación C: diseño e implementación de programas. Félix García, Jesús Carretero, Javier Fernández y Alejandro Calderón. Prentice-Hall, 2002.
- N. Matthew and R. Stones. Programación Linux. Anaya Multimedia. 2008. ISBN: 978-8441524422
- Lenguaje C. https://en.cppreference.com/w/c
- POSIX. https://pubs.opengroup.org/onlinepubs/9799919799/functions/contents.html
- Sistemas Operativos: Una visión aplicada. Jesús Carretero, Félix García, Pedro de Miguel y Fernando Pérez. McGraw-Hill, 2001.
- Unix man pages (man function)